

# The dBTC Is the Asset; the Bitcoin Key Is Machinery

dBTC: A High-Level Explainer for Bitcoin Readers

DSM-NATIVE dBTC V1 — consumption-gated release, successor vaults, and no second double-spend system

Brandon “Cryptskii” Ramsay  
Irrefutable Labs Inc.

## Abstract

dBTC is Bitcoin-backed economic state native to the Deterministic State Machine. A real Bitcoin output is funded under a vault profile and verified at the required depth; exactly that quantity is admitted as dBTC into DSM state. From then on dBTC moves the way every other DSM asset moves: as ordinary bilateral transitions, online or offline, with no Bitcoin transaction, no confirmation, no depositor, no custodian, and no ledger.

The central rule is that possession of Bitcoin-related bytes is not possession of dBTC. A party may hold the vault identifier, the lineage, every public receipt, the encrypted execution capsule, the fulfillment hash, and every storage replica, and still have no withdrawal authority. Withdrawal begins on the DSM side: the holder proves live dBTC state under authority it can satisfy and consumes it in a burn bound to one exact withdrawal intent. That consumption produces completion evidence, the completion evidence derives the vault’s fulfillment secret, and only then does constrained Bitcoin execution authority open — for the one committed transaction and nothing else.

A full withdrawal consumes the dBTC and terminates the backing generation. A partial withdrawal spends the current backing output into an external payout plus a *successor vault* holding the remainder under fresh execution authority, in the same immutable origin lineage.

The consequence is that dBTC needs no second double-spend system. DSM has already answered whether the state exists and whether it may be consumed; the vault only makes Bitcoin release conditional on that answer. No federation, signing quorum, validator set, mint, or global dBTC ledger is introduced, and storage nodes remain librarians rather than judges.

This explainer accompanies the DSM-NATIVE dBTC V1 specification [1] and the DSM and SoFi papers it builds on [2, 3, 4, 5]. It is written for a Bitcoin reader and organised around that reader’s objections rather than the order in which the mechanism is constructed. Shaded boxes state how Bitcoin, Lightning, eCash, statechains, and wrapped Bitcoin solve the corresponding problem and where this differs. Costs and boundaries are stated where they arise.

# Contents

|     |                                                    |    |
|-----|----------------------------------------------------|----|
| I   | What Is Claimed, and Who Decides                   | 4  |
| 1   | The Central Question                               | 4  |
| 2   | The Entire Idea in One Diagram                     | 5  |
| 3   | Six Answers Before the Details                     | 6  |
| 4   | What a dBTC Transfer Is                            | 7  |
| 5   | No Separate Double-Spend System                    | 8  |
| 6   | Withdrawal Is a Consumption First                  | 9  |
| 7   | Knowledge Is Not Possession                        | 10 |
| 8   | Storage Nodes Are Librarians                       | 11 |
| 9   | The Depositor Never Comes Back                     | 12 |
| II  | The Construction                                   | 12 |
| 10  | Origin: Where dBTC Comes From                      | 12 |
| 11  | The Asset and Its Provenance                       | 13 |
| 12  | The Burn                                           | 14 |
| 13  | From Completion Evidence to the Fulfillment Secret | 14 |
| 14  | Why the Bitcoin Key Is Sealed                      | 15 |
| III | Getting Out                                        | 16 |
| 15  | Full Withdrawal                                    | 16 |
| 16  | Partial Withdrawal and the Successor Vault         | 16 |
| 17  | Fees and Conservation                              | 18 |
| 18  | Crash Safety                                       | 19 |
| IV  | Domains, Compromise, and Assessment                | 19 |
| 19  | Online, Offline, and the SoFi Boundary             | 20 |
| 20  | What a Device Compromise Means                     | 21 |

|    |                                                 |    |
|----|-------------------------------------------------|----|
| 21 | Security Properties                             | 21 |
| 22 | Relation to Everything Else That Works          | 21 |
| 23 | What Is Not Introduced, and What Is Not Claimed | 22 |
| 24 | The Complete Causal Architecture                | 23 |
| 25 | Final Invariant                                 | 23 |
| 26 | Conclusion                                      | 24 |

## Part I

# What Is Claimed, and Who Decides

The Bitcoin reader’s objections, in the order they occur. Everything here is stated normatively in the specification; the mechanism is built in Part II.

## 1 The Central Question

Every design that puts Bitcoin somewhere else has to answer one question:

What exactly do I have to hold in order to get the Bitcoin back, and what stops someone else who holds bytes from getting it instead?

Most answers name an authority. A custodian’s database says you own it. A mint’s signature says you own it. A federation’s threshold says so. A statechain entity’s key share says so. In every case, the thing that decides ownership is not the thing you hold.

DSM-NATIVE dBTC V1 answers differently:

You hold live dBTC state in DSM, under authority you can satisfy. Nobody else holding anything at all can get the Bitcoin, because the vault will not release until that state has been consumed, and only you can consume it.

### The rule everything follows from

Bitcoin bytes  $\nRightarrow$  dBTC ownership.

A party may know the vault identifier, the full lineage, every public receipt, the ciphertext of the execution capsule, the public execution key, the fulfillment hash, and hold a copy of every storage record. None of that is ownership, and none of it produces a withdrawal, because a withdrawal begins with the consumption of real dBTC state and there is nothing valid to consume.

The positive form is a conjunction, and every term is required:

live dBTC  $\wedge$  valid authority  $\wedge$  valid consumption  $\wedge$  DLV fulfillment  $\implies$  authorised Bitcoin release.

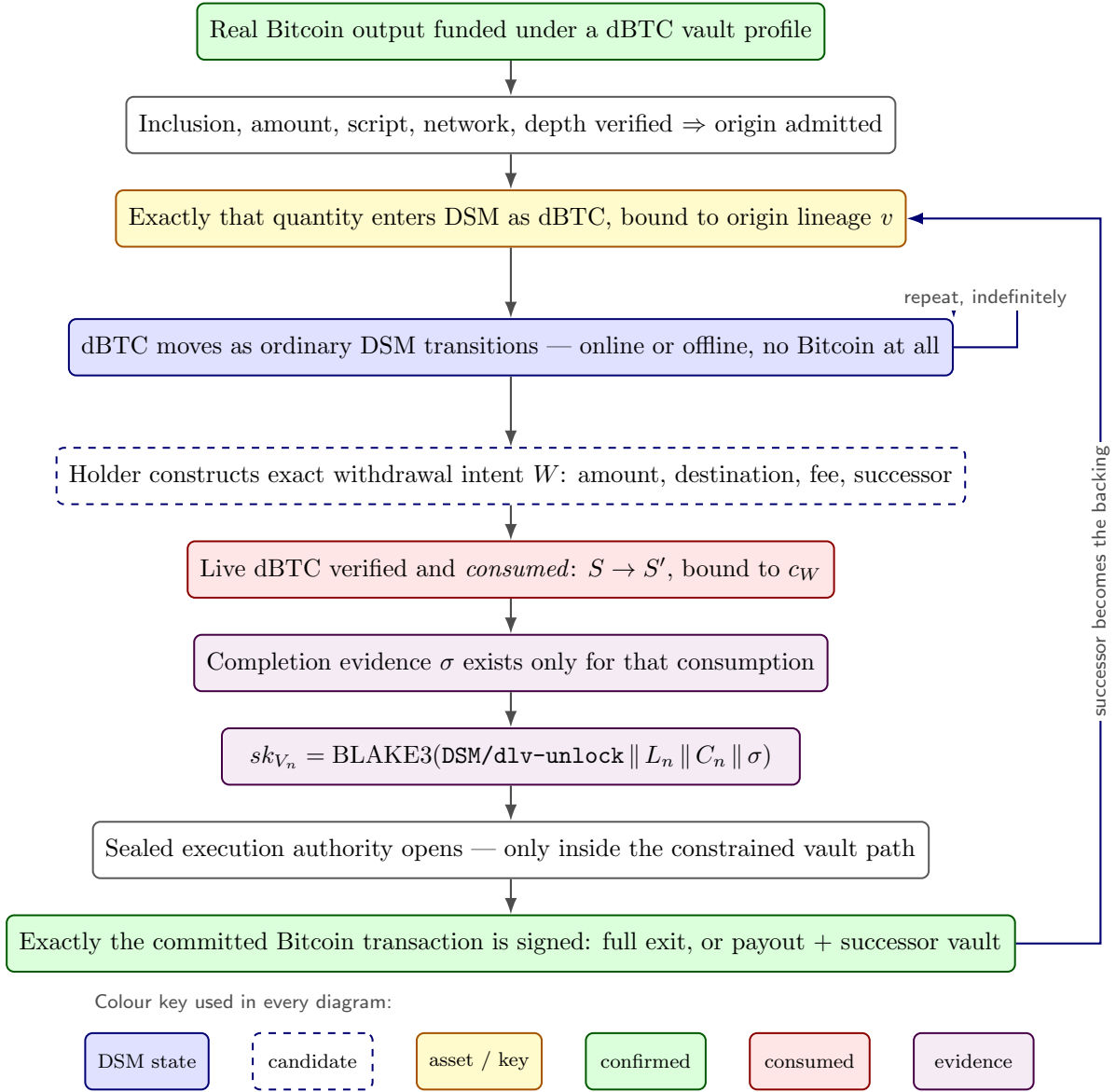
### Two authorities, never conflated

**Economic authority** is ownership of live DSM dBTC state. It is what transfers, what is proved, what is consumed. It is the asset.

**Execution authority** is the narrowly scoped Bitcoin material needed to realise one authorised withdrawal. It is settlement machinery. It is not evidence of ownership, it is not transferable bearer state, and it is not the thing anyone holds between transfers.

Confusing the two is the failure mode this specification exists to prevent, and Requirement 2.3 forbids it outright: no implementation may treat a vault identifier, lineage, ciphertext, public key, hash lock, preimage commitment, storage record, or transaction template as sufficient evidence of ownership.

## 2 The Entire Idea in One Diagram



Two features of that picture are the whole design.

The middle band is wide and touches nothing. dBTC circulates entirely inside DSM. There is no Bitcoin transaction per transfer, no confirmation to wait for, no fee per hop, no relay dependency, no lineage of pre-signed spends to carry, and no dust floor on what you can send.

The bottom band runs one way only. Every arrow from the intent down to the signature is a consequence of the one above it. You cannot enter the chain partway up: there is no path to the signature that does not begin with a consumed dBTC state.

### 3 Six Answers Before the Details

| The question the reader has                       | The answer, and where it is earned                                                                                                                                                                                                                                                                      |
|---------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| What do I actually hold?                          | Live dBTC state in your own DSM state, under authority you can satisfy, carrying the origin lineage it traces to. Not a key, not a preimage, not a receipt. (Definition 2.1; Section 11)                                                                                                                |
| Does a transfer touch Bitcoin?                    | No. A dBTC transfer is an ordinary DSM economic transition: your state is consumed, the recipient is credited, provenance is preserved. No Bitcoin confirmation, no depositor, no custodian, no quorum, no mint, no global ledger, no storage-node verdict. (Requirement 10.1; Section 4)               |
| What stops a double spend, then?                  | DSM does, exactly as it does for every other asset: hash-adjacent state, consumed parents, Tripwire, conservation. dBTC adds no second ownership system, and the vault does not re-solve the problem — it makes Bitcoin release conditional on DSM’s answer. (Section 5)                                |
| Then what stops someone with all the vault bytes? | Nothing about them is ownership. Release requires a valid burn of live dBTC bound to one exact intent; the fulfillment secret is derived from that burn’s completion evidence and from nothing else. Copied lineage, copied ciphertext, copied preimage commitment, stale state: all inert. (Section 7) |
| What happens if I only want part of it out?       | The backing output is spent into your payout plus a successor vault holding the remainder under <i>fresh</i> execution authority, in the same origin lineage. The spent parent is terminal; parent and successor are never both live backing. (Section 16)                                              |
| Who has to be online?                             | You. Not the original depositor, not an issuer, not a signer, not a storage node with any authority. Storage may hold vault material so a future holder can fetch it; unavailability delays retrieval and creates nothing. (Sections 9, 8)                                                              |

#### Just enough notation

**Origin, vault, generation.** A dBTC origin is a confirmed Bitcoin output admitted under a vault profile. The vault identifier  $v$  is stable forever; the *backing generation*  $n$  and the backing outputpoint  $o_n$  change when the backing is actually spent.  $V_n$  is the vault at generation  $n$ .

**Provenance.** A holder’s dBTC is a set of origin allocations  $\{(v_i, q_i)\}$ : quantity  $q_i$  traceable to origin  $v_i$ . The wallet may show one balance  $Q = \sum q_i$ ; the protocol keeps the provenance so withdrawals reconcile against real backing.

**Withdrawal intent  $W$ .** The exact thing being asked for: origin, dBTC quantity consumed, external payout, fee treatment, destination, backing generation and outputpoint, source state, and the successor rule if partial. Its commitment is  $c_W$ .

**Burn.** The DSM transition  $S \xrightarrow{\text{Burn}(W)} S'$  that consumes spendable dBTC and produces a non-spendable withdrawal-completion object.

**Completion evidence  $\sigma$ .** The canonical DSM proof-of-completion commitment for that burn, binding both the transition and the exact intent.

**Lock, parameters, secret.**  $L_n$  is the vault’s fulfillment condition,  $C_n$  its parameter commitment. The unlock secret is  $sk_{V_n} = \text{BLAKE3-256}(\text{DSM}/\text{dlv-unlock} \parallel \text{CCB}(L_n) \parallel C_n \parallel \sigma)$ , and the vault commits  $h_{f,n} = \text{SHA256}(sk_{V_n})$ .

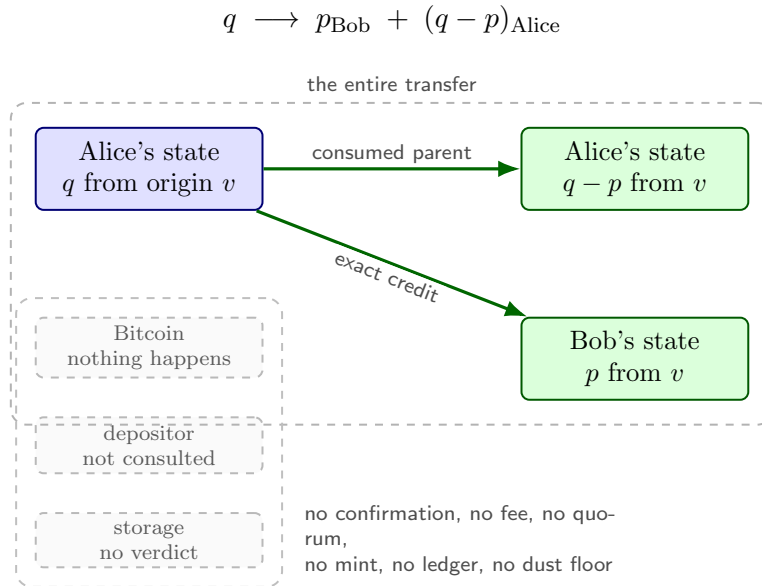
**Capsule  $E_n$ .** The sealed Bitcoin execution scalar for generation  $n$ . Storage may hold it. Holding it grants nothing.

**Class C, K, N.** Verifier; SDK that constructs and orchestrates; storage node, non-authoritative.

## 4 What a dBTC Transfer Is

Alice owns  $q$  of dBTC and pays Bob  $p$ , where  $0 < p \leq q$ .

That is a DSM transition and nothing else. Alice's predecessor economic state is consumed; Bob is credited exactly  $p$  from a canonical source; Alice keeps exactly  $q - p$ ; provenance is preserved on both sides; the consumed source cannot remain simultaneously spendable. The existing DSM transfer verifier proves all of it.



### What this buys

Because a transfer is pure DSM state, the constraints a Bitcoin layer usually inherits are simply absent. There is no per-hop fee, because nothing is being relayed. There is no dust floor on a transfer, because the quantity is DSM state in satoshi-equivalent units, not an output that must survive on its own. There is no unsettled depth to bound and no chain of pre-signed spends to carry. There is no waiting.

And the transfer works in whichever DSM custody domain the holder is using, including offline, because it is the same machinery as any other DSM economic transition.

#### Key Idea

A successor Bitcoin vault is *not* created because ownership changed. The backing generation advances only when the backing outpost is actually spent, which happens at withdrawal — and most importantly at a partial withdrawal.

#### Bitcoin Comparison

A Bitcoin payment is a transaction that must be relayed, fee'd, mined, and confirmed. A Lightning payment needs a channel, liquidity, a route, and a live counterparty. An eCash payment is a bearer

token the mint must redeem. A statechain or Ark payment needs the entity or operator in the loop. A dBTC transfer is a DSM state transition between two parties. The chain is not a participant and does not learn that it happened.

**Architectural consequence:**

Everything Bitcoin charges for per transaction — fees, confirmation latency, relay policy, dust minimums — does not apply to a transfer, because a transfer is not a transaction. Bitcoin’s role is collateral and final settlement, not bookkeeping.

**Tradeoff / boundary:**

The backing is only as good as the origin admission and the vault profile that governs release. And the security of your dBTC is the security of your DSM custody, no better: Section 20 states that plainly.

## 5 No Separate Double-Spend System

The reader’s next question is the right one: if transfers never touch Bitcoin, what stops Alice paying Bob and Carol the same dBTC?

DSM does. Not a bridge, not a vault, not a register invented for this asset. The withdrawal verifier asks the same question DSM already asks of every economic transition:

Does this party possess a valid current state, under valid authority, and does the proposed transition consume that state according to the committed rules?

Accepted state is hash-adjacent to its predecessor rather than ordered by a global sequence. A realized transition consumes its identified parent. Conflicting accepted successors to one consumed state are excluded by Tripwire. Debits, credits, and provenance pass only through canonical transitions satisfying conservation. Those four facts are the substrate, and dBTC imports them rather than duplicating them.

The consequence for the bridge is a chain of implications with no way in from the side:

$$\neg \text{ValidBurn}(W, \sigma) \Rightarrow \neg \text{VerifyCompletion}(\sigma) \Rightarrow \neg sk_{V_n} \Rightarrow \neg \text{Unlock}(V_n).$$

No valid burn, no completion evidence. No completion evidence, no unlock secret. No unlock secret, no Bitcoin release. Producing arbitrary dBTC-looking bytes does not help, because the claimant must prove actual state inclusion, current-state validity, authority, provenance, and consumption. And a prior owner who kept a copy of an old state does not regain anything after that state has been consumed by an accepted successor.

### Key Idea

The vault does not solve double spending a second time. DSM has already answered whether the state exists and whether it may be consumed. The vault only makes Bitcoin release conditional on that answer.

### Bitcoin Comparison

Bitcoin prevents double spends with global ordering: one chain, one history, every node revalidating. Every bridge built on top has then needed its own answer for the value while it is off-chain — a mint’s database, an entity’s key share, a channel’s revocation, an operator’s signature. dBTC needs no such answer because DSM already is one, and dBTC is an ordinary DSM asset.

**Architectural consequence:**



The bridge introduces no consensus mechanism, no validator set, no global dBTC ledger, and no second double-spend database. Its entire job is to gate one Bitcoin spend on a verification DSM was performing anyway.

**Tradeoff / boundary:**

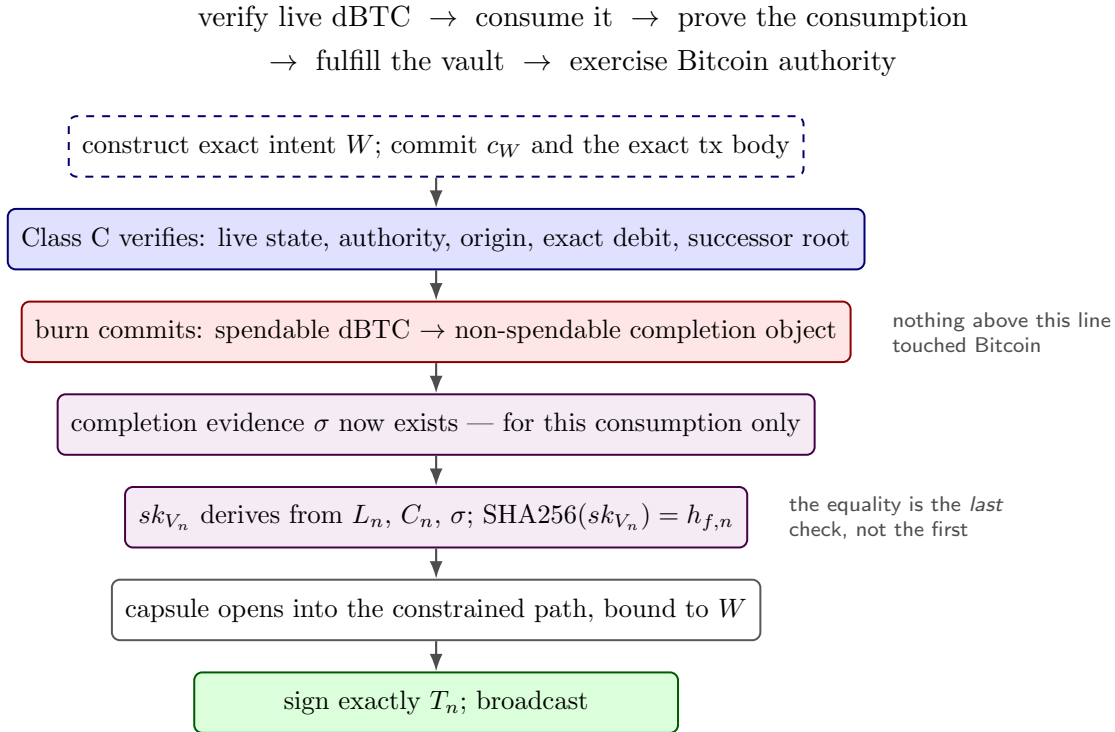
That inheritance runs both ways. dBTC is exactly as safe as DSM ownership and no safer; adding Bitcoin collateral does not strengthen the state machine, and it must not be allowed to weaken it by becoming an independent bearer-authority system.

## 6 Withdrawal Is a Consumption First

This is the boundary the whole specification turns on, and it is worth stating as an ordering, because the intuitive order is the wrong one.

The intuitive order is: get the key, spend the Bitcoin, then burn the dBTC you promised to burn. That order requires trusting the promise, and it is how a bridge acquires a custodian.

The actual order is the reverse:



Two things follow that a reader should hold onto.

First, the burn is not a promise that gets settled later. It is a completed DSM transition, and the consumed dBTC is not ordinary spendable dBTC after it. A burn proof is not accepted because its bytes are well formed; Class C verifies the actual transition, including source inclusion, freshness, authority, provenance, exact debit, and successor root.

Second, the intent is exact. Changing the amount, destination, origin, backing outpoint, fee treatment, or successor construction changes  $c_W$ , so a burn authorises one withdrawal and no other. The execution routine verifies that the candidate transaction equals the transaction the burn committed to; authority released by one burn cannot pay an arbitrary destination or amount.

## Bitcoin Comparison

In a custodial or federated bridge the redemption request is a message, and the honest behaviour of the signer is what makes it true. In a deleted-key construction the payer’s deletion is what you have to believe.

Here the withdrawal begins as a state transition the verifier checks, and the Bitcoin secret is derived from that transition’s evidence. There is no promise, and nothing to believe about anyone’s behaviour.

### Architectural consequence:

The unlock secret cannot exist before the consumption exists. The cryptographic equality  $\text{SHA256}(sk_{V_n}) = h_{f,n}$  is necessary but never independently sufficient at the DSM layer: it is the final check of an already-verified state relation.

### Tradeoff / boundary:

Withdrawal is online by design, and it depends on Bitcoin’s own settlement assumptions — the confirmation depth chosen in the vault profile, and no claim that a deeper reorganisation is impossible.

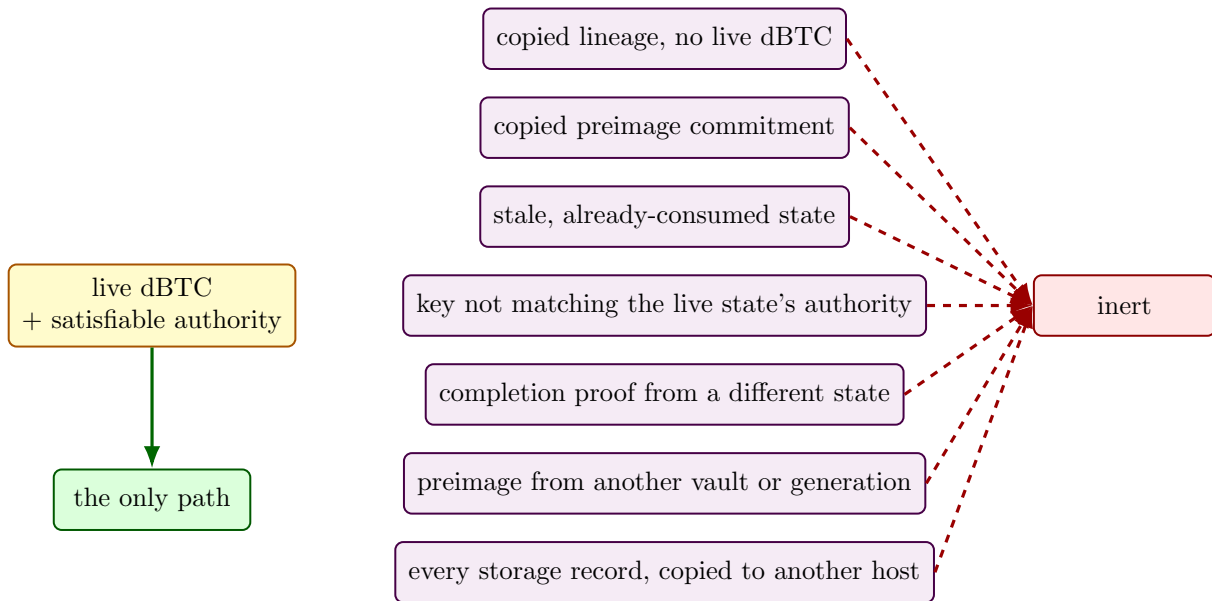
## 7 Knowledge Is Not Possession

This deserves its own section because it is where a Bitcoin reader’s instincts are most likely to mislead. On Bitcoin, knowledge of a private key *is* ownership. On dBTC it is not, and the vault is built so that it cannot be.

Suppose an adversary has all of it: the complete vault descriptor, the complete dBTC lineage, every public DSM receipt, the entire Bitcoin transaction history, the encrypted execution capsule, the public execution key, the fulfillment hash commitment, and a copy of every storage replica.

None of that creates a valid burn, and the withdrawal relation is conjunctive:

$$\text{Withdrawable} = \text{LiveState} \wedge \text{StateAuthority} \wedge \text{ValidBurn} \wedge \text{ValidCompletion} \wedge \text{Preimage}.$$



The reason is structural rather than a policy someone enforces. The fulfillment secret is derived from the lock, the parameters, and the completion evidence of one specific burn. Substituting any of the three changes the derivation except with negligible probability. There is no value an adversary can compute, copy, or replay that stands in for a consumption that never happened.

## Bitcoin Comparison

“Not your keys, not your coins” is exactly right on Bitcoin, where the key is the asset.

On dBTC the sentence would be “not your live state, not your coins.” The Bitcoin key is machinery: sealed, opened only downstream of verification, usable only for the transaction the burn committed to, and useless to anyone who obtains its ciphertext.

### Architectural consequence:

Because the asset is state rather than a secret, copying does not steal it, publishing does not weaken it, and storing the vault material publicly is safe enough to be the default — which is what lets a future holder withdraw without ever meeting the depositor.

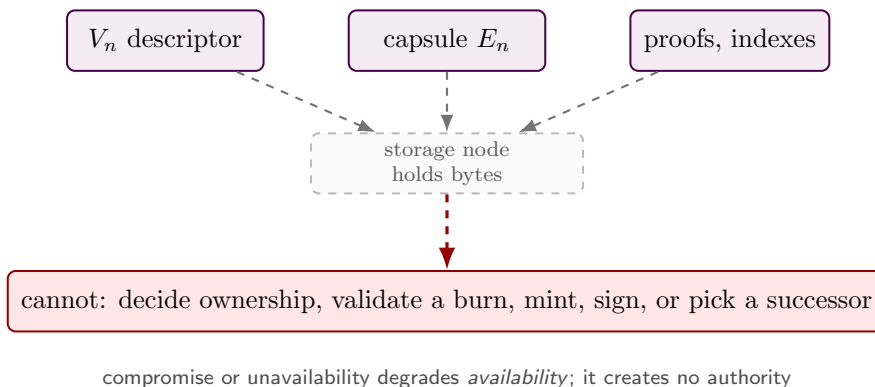
### Tradeoff / boundary:

The converse of “knowledge is not possession” is that possession is whatever DSM says it is. An attacker who compromises enough of your DSM authority to produce a transition the verifier accepts has compromised that dBTC, and the vault will not save you (Section 20).

## 8 Storage Nodes Are Librarians

Storage exists for one reason: so that a dBTC holder years from now can retrieve the vault material even though the original depositor is permanently gone. For each vault generation a node may hold the descriptor, the sealed capsule, proofs, inclusion evidence, and indexes.

What it must not do is decide anything. A storage node must not determine who owns dBTC, validate a burn as an economic authority, hold a mint key, hold plaintext reusable Bitcoin vault keys, sign a Bitcoin exit, convert an invalid DSM transition into a valid one, or choose a successor on the protocol’s behalf. Storing encrypted vault bytes does not make a node a signing committee, and the specification says so explicitly, because that is exactly the drift a bridge design tends toward.



A malicious storage service can withhold, and a dead one can be unreachable. Both are availability failures. Neither produces a valid burn, and copying every record to another host creates no dBTC.

## Bitcoin Comparison

A wrapped-Bitcoin custodian’s database *is* the ledger. A federation’s signers *are* the authority. Even in designs that minimise trust, the infrastructure holding the data usually holds something that matters.

Here the infrastructure holds ciphertext and proofs, and holding them is worth nothing.

### Architectural consequence:

The set of parties who could steal dBTC by being compromised does not include anyone who runs infrastructure. It includes only the holder.

**Tradeoff / boundary:**

Availability is a real dependency for withdrawal: if nobody can serve the vault material, a holder may be unable to construct a withdrawal until it is retrieved from somewhere. Holders and implementations should keep their own copies rather than relying on any one service.

## 9 The Depositor Never Comes Back

After a valid origin admission, the party who funded the Bitcoin has no continuing role. They do not sign withdrawals, do not approve transfers, and do not need to exist.

A later bearer obtains the lineage, the current vault generation, the sealed capsule, the Bitcoin inclusion proof, the public parameters, and the DSM proof material from any available source — storage, a counterparty, their own archive — and withdraws on the strength of their own dBTC state.

**Key Idea**

Issuer non-liveness is not a feature bolted on. It falls out of the fact that the release condition is a property of the withdrawer's own state, not a decision by anyone else.

**Bitcoin Comparison**

Lightning needs your counterparty responsive or your watchtower awake. Ark needs the operator for rounds and refreshes. Statechains need the entity. eCash needs the mint to redeem. Wrapped Bitcoin needs the custodian to process your request.

dBTC needs you.

**Architectural consequence:**

There is no party whose disappearance strands the asset, and no party whose refusal blocks a redemption.

**Tradeoff / boundary:**

Bitcoin itself still has to be reachable to broadcast, and the vault material still has to be retrievable from somewhere. Neither is an authority; both are availability.

## Part II

# The Construction

Origin admission, the asset and its provenance, the burn, how the fulfillment secret is derived, and why the Bitcoin key is sealed rather than held.

## 10 Origin: Where dBTC Comes From

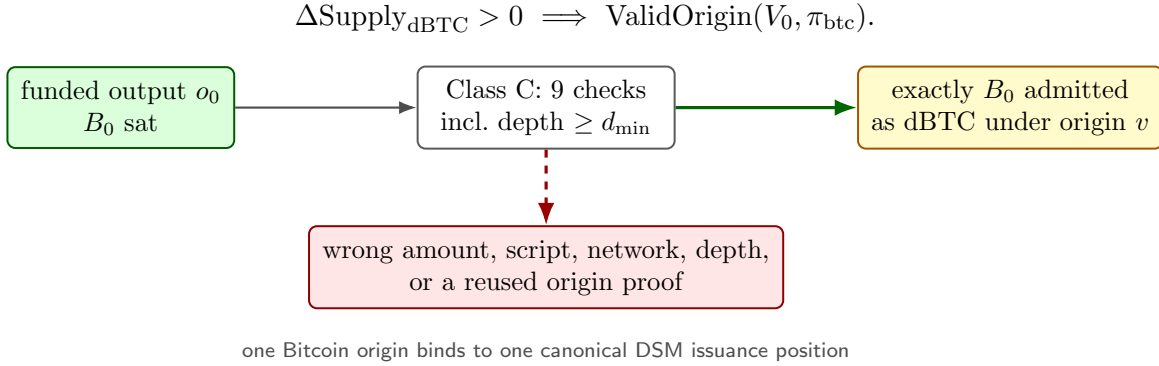
A dBTC origin begins with a real Bitcoin output  $o_0 = (\text{txid}_0, \text{vout}_0)$  of amount  $B_0$ , paid to a script committed to a dBTC vault profile. Abstractly the origin vault is

$$V_0 = (v, 0, o_0, B_0, L_0, C_0, P_0, h_{f,0}, \Pi_{\text{btc}}, \Pi_{\text{policy}})$$

— the stable vault identifier, the backing generation, the outpoint and amount, the fulfillment condition and its parameter commitment, the Bitcoin execution public key, the committed fulfillment hash, the Bitcoin network and confirmation profile, and the dBTC policy and successor rules.

A Bitcoin output does not create dBTC because a client says it exists. Class C verifies nine things before any issuance: the funding transaction is valid under the configured Bitcoin verifier; the claimed outpoint exists in it; the amount equals  $B_0$ ; the output script matches the committed vault profile; the transaction is proven included in an accepted chain; the required confirmation depth is met; the origin identifier recomputes from canonical origin material; the exact Bitcoin origin is bound to one canonical DSM issuance position; and the policy commitment matches the canonical dBTC asset.

The mint gate is then a single implication:



### Bitcoin Comparison

Minting on a custodial bridge is the custodian saying it received coins. Minting on a federated bridge is a threshold of signers saying so.

Minting dBTC is a verifier checking Bitcoin itself: the transaction, the inclusion, the depth, the exact script, the exact amount.

**Architectural consequence:**

No party issues dBTC. Provenance is the deposit, and no self-asserted mint or custom credit source can manufacture a positive balance.

**Tradeoff / boundary:**

Origin admission depends on the confirmation depth in the vault profile. A reorganisation deeper than that depth is an external Bitcoin assumption, and DSM does not claim to prevent it.

## 11 The Asset and Its Provenance

One dBTC base unit is one satoshi-equivalent unit of DSM economic state whose positive issuance traces to accepted Bitcoin backing. The asset identifier alone is not value; a valid quantity requires provenance.

A holder's dBTC is therefore a set of origin allocations

$$A = \{(v_i, q_i)\}_{i=1}^m, \quad Q = \sum_{i=1}^m q_i,$$

where  $q_i$  is the outstanding quantity attributable to origin lineage  $v_i$ . The wallet may present  $Q$  as one fungible balance; underneath, the protocol keeps what it needs to reconcile a withdrawal against real backing.

### Multi-origin withdrawals

A holder with  $\{(v_A, 7000), (v_B, 3000)\}$  may withdraw 10,000, and the Bitcoin transaction then has two backing inputs. Each input must be independently justified by dBTC consumed from its own origin. The invariant is per-origin:

$$\text{BurnFrom}(v_i) \leq \text{LiveDbtcFrom}(v_i)$$

before the burn. dBTC attributed to one origin may not be redeemed against an unrelated origin's backing unless an explicit canonical transition has changed the provenance relationship.

### Key Idea

Fungible to the user, provenanced to the protocol. That is what allows one asset with many independent Bitcoin backings and no shared pool.

## 12 The Burn

A withdrawal burn is a canonical DSM transition consuming actual spendable dBTC and producing a non-spendable withdrawal-completion object. Class C must reject it unless the claimant possesses the live state and satisfies the authority that state requires, and it must verify the transition itself rather than the well-formedness of a proof.

Conceptually  $\text{ValidBurn}(W, \sigma)$  requires all of:

| Condition                                           | What it rules out                                     |
|-----------------------------------------------------|-------------------------------------------------------|
| $\text{Live}(S)$                                    | withdrawing against a state already consumed          |
| $\text{Own}(\text{claimant}, S)$                    | withdrawing against someone else's state              |
| $\text{Origin}(S, a) = v$                           | redeeming one origin's dBTC against another's backing |
| $\text{Valid}(S \rightarrow S')$                    | a transition DSM would not accept                     |
| $\text{Consumed}(S, a)$                             | keeping the dBTC and taking the Bitcoin               |
| $\text{IntentCommit}(S \rightarrow S') = c_W$       | a burn that authorises a different withdrawal         |
| $\text{VerifyCompletion}(\sigma, S \rightarrow S')$ | evidence from some other transition                   |

### Key Idea

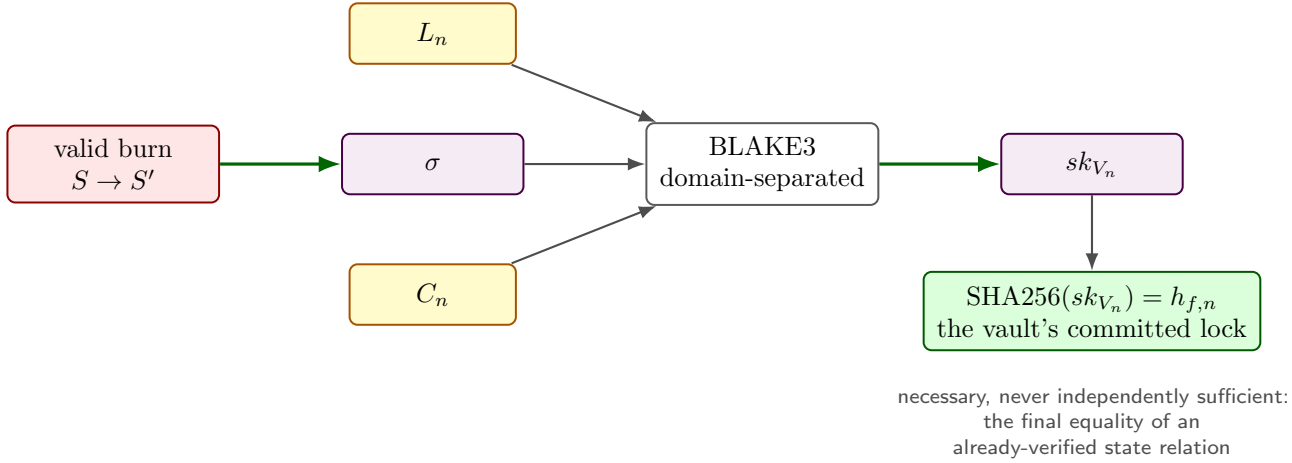
The scarce object is the live dBTC state. The preimage does not create the burn; the burn creates the completion evidence from which the correct fulfillment secret can be obtained.

## 13 From Completion Evidence to the Fulfillment Secret

Let  $\sigma$  be the canonical completion commitment of a valid burn, binding both the state transition that consumed the dBTC and the exact withdrawal intent. The vault's unlock value is derived from the lock, the parameters, and that evidence:

$$sk_{V_n} = \text{BLAKE3-256}(\text{DSM/dlv-unlock} \parallel \text{CCB}(L_n) \parallel C_n \parallel \sigma), \quad h_{f,n} = \text{SHA256}(sk_{V_n}).$$

A candidate  $sk_{V_n}$  is valid only if nine things align: the live dBTC state, the claimant's authority, the consumed quantity, the origin lineage, the withdrawal intent, the lock  $L_n$ , the parameter commitment  $C_n$ , the completion commitment  $\sigma$ , and the committed fulfillment hash  $h_{f,n}$ .



Substituting any of  $L_n$ ,  $C_n$ , or  $\sigma$  changes the derivation except with negligible probability. A burn proof from another vault fails. A burn proof from another generation fails. Completion evidence that does not bind the exact intent fails.

## 14 Why the Bitcoin Key Is Sealed

The Bitcoin execution scalar  $x_n$ , with  $P_n = x_n G$ , is not the bearer asset and must not sit in transferable dBTC state as an ordinary reusable secret. It lives outside that state as a sealed capsule

$$E_n = \text{SealVaultAuthority}(x_n; v, n, L_n, C_n, P_n),$$

which storage nodes may freely persist. Possession of  $E_n$  alone provides no usable signing authority. It opens only on a valid burn and a matching preimage:

$$\text{OpenVaultAuthority}(E_n, sk_{V_n}, \sigma, W) \rightarrow A_n \quad \text{only if} \quad \text{ValidBurn}(W, \sigma) \wedge \text{SHA256}(sk_{V_n}) = h_{f,n}.$$

And what opens is *constrained* authority. A conforming high-assurance implementation must not expose  $x_n$  through a general-purpose signing or export interface after fulfillment: the authority is usable only by the vault execution path bound to the verified intent.

### Key Idea

This requirement exists for one specific attack: a valid *partial* burn must not become an unrestricted ability to sweep the rest of the vault. The protocol is not “preimage means Bitcoin key.” It is: valid burn  $\rightarrow$  valid completion  $\rightarrow$  matching preimage  $\rightarrow$  constrained execution of the committed transaction.

### The Bitcoin side

The current profile may use the existing dual-hashlock vault script. The fulfill branch requires the hash equality  $\text{SHA256}(sk_{V_n}) = h_{f,n}$ , plus a signature under  $P_n$ . Both conditions must correspond to the same generation — a preimage from one generation may not be paired with execution authority from another.

If a refund branch exists, it must not be a discretionary depositor backdoor. No refund secret may be exposed because a clock expired or a depositor asked; any refund must derive from a mutually exclusive DSM condition proving the refund branch is valid under the committed policy, and backing must never leave while corresponding live dBTC remains outstanding.

## Transaction binding

Class K constructs the exact candidate transaction  $T_n$  before any constrained execution, and the intent binds its commitment. The execution routine verifies that the candidate equals the committed transaction. Authority released by one burn cannot authorise an arbitrary destination or amount.

### Bitcoin Comparison

In an HTLC, knowing the preimage is enough — that is the point, and it is why HTLC-based bridges have to be careful about who learns it and when.

Here the preimage is not a token that circulates. It is a value that only exists downstream of a verified consumption, and knowing it still does not let you sign anything other than the transaction the burn committed to.

#### Architectural consequence:

The hashlock is the last check rather than the first, and the key it unlocks is scoped to one transaction. Neither the preimage nor the ciphertext is worth stealing.

#### Tradeoff / boundary:

“Must not expose the scalar through a general-purpose interface” is an implementation obligation, not a theorem. An implementation that exports a reusable key after a partial burn is non-conforming, and the specification declines to claim otherwise.

## Part III

# Getting Out

Full withdrawal, partial withdrawal and the successor vault, conservation, and what happens when the power fails halfway through.

## 15 Full Withdrawal

A full withdrawal consumes all the dBTC backed by the claimant’s selected vault quantity and leaves no successor for that backing. With the fee paid from the backing,

$$B_n = p + f,$$

the DSM side moves  $\text{Spendable}(B_n) \rightarrow \text{Burned}(B_n)$  and the Bitcoin side spends  $V_n(B_n)$  into the destination payout plus fee. After the backing outpoint is validly spent, that generation is terminal and must not advertise a live successor.

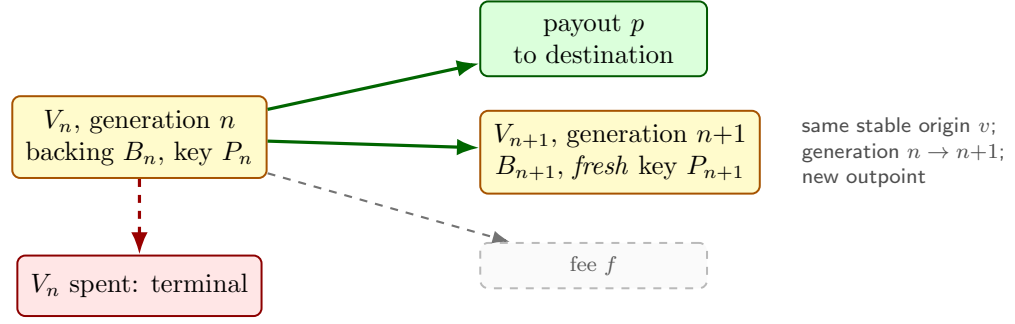
## 16 Partial Withdrawal and the Successor Vault

This is the case that makes the design interesting, and the one where a naive implementation would go wrong.

The holder has dBTC backed by  $V_n(B_n)$  and wants only  $p$  out, with fee  $f$  charged to that backing, so  $a = p + f$  and  $0 < a < B_n$ . The Bitcoin transaction is

$$V_n(B_n) \longrightarrow \text{BTC}_{\text{destination}}(p) + V_{n+1}(B_{n+1}) + \text{fee}(f), \quad B_{n+1} = B_n - p - f.$$





$$B_n = p + B_{n+1} + f + \delta_A \quad \text{under the declared transaction profile}$$

## The successor keeps the origin and changes everything else

The vault identifier is stable:  $v_{n+1} = v_n = v$ . The remaining dBTC still traces to the same origin lineage, and a recipient verifying provenance sees continuity. What changes is the backing: a new generation, a new outpoint, and fresh generation-specific fulfillment material  $L_{n+1}$ ,  $C_{n+1}$ ,  $h_{f,n+1}$ ,  $E_{n+1}$ .

## Fresh authority is mandatory

$$x_{n+1} \neq x_n, \quad P_{n+1} = x_{n+1}G.$$

A partial withdrawal must not return the remainder to the spent parent execution key. The successor derivation must be canonical, domain-separated, and pinned by known-answer vectors, binding at minimum the stable vault identifier, the successor generation, the parent outpoint, the exact transaction commitment, the successor vault parameters, and fresh successor derivation material. The parent scalar alone must not constitute a reusable successor credential outside the constrained transition.

## The successor cannot be left as dust

A reader who has followed this far will ask what stops a withdrawer from draining a vault down to a successor worth a few hundred satoshis. The answer is the same one DSM gives everywhere else: the split is not a candidate the state machine will accept.

The floor is committed at origin, in  $\Pi_{\text{policy}}$ , alongside the rest of the vault's rules. Write it  $B_{\min}$ . It is immutable across every generation of the vault, so it is a property of the lineage rather than of whoever happens to be withdrawing, and any verifier re-checking a completed withdrawal later can confirm it was respected.

The check happens at the burn, not at transaction construction, and the distinction matters. Class K builds transactions; Class C decides validity. A construction-time check is advice. A burn predicate is the protocol:

$$B_{n+1} = B_n - p - f \geq B_{\min}$$

evaluated as one more term of the burn's acceptance alongside conservation, provenance, authority, and exact debit. The intent  $W$  already binds  $p$ ,  $f$ , the generation, the outpoint, and the successor rule, so the predicate reaches for nothing it does not already have.

If it fails, the burn is not accepted. And because nothing downstream exists without an accepted burn, the refusal is total rather than partial:

$$\neg \text{ValidBurn} \Rightarrow \neg \sigma \Rightarrow \neg sk_{V_n} \Rightarrow \text{no signature, no transaction, nothing to unwind.}$$

## Key Idea

The remainder is never trimmed into fee. In a design where the remainder is the withdrawer's own value, trimming is harmless — they are burning their own money. Here the successor is collateral for outstanding dBTC that the withdrawer may not hold, so destroying it would take value from someone else. The split is refused instead.

Refusing costs the withdrawer nothing structural. dBTC is fungible to the user and provenanced to the protocol, so a holder whose allocation spans several origins draws against whichever active vaults admit the split. A vault that cannot be partially withdrawn without breaching its floor is still fully withdrawable by the holder of its remaining dBTC.

## Why the parent cannot remain the authority

A partial withdrawal consumes  $o_n$ . After the split,  $o_n$  is not the backing object; the remainder is represented only by  $V_{n+1}$ . The protocol must not represent both as active backing:

a spent parent generation and its confirmed successor are never both live backing.

That is the Bitcoin-side analogue of ordinary DSM parent consumption, and it is the same rule the state machine applies to everything else.

## Activation

Before the transaction reaches the required depth, the successor is *pending*; after its output is proven at depth, it becomes *active*. Activation changes the backing generation, not the economic origin.

## Bitcoin Comparison

A Lightning channel's partial spend is a new commitment transaction both parties sign, revoking the old one, with a penalty if you publish the stale state. A statechain transfers the whole UTXO or nothing.

A dBTC partial withdrawal spends the backing into your payout and a fresh vault, in one transaction, with no counterparty and no revocation — the old generation is dead because its outpoint is spent, which Bitcoin already enforces.

### Architectural consequence:

Partial exit at arbitrary granularity with no channel, no operator round, no coin lifetime, and no timelock; and the remaining dBTC keeps circulating under the same origin without anyone re-issuing it.

### Tradeoff / boundary:

The withdrawal transaction is a real Bitcoin transaction and pays a real fee, and the successor is not usable as backing until it confirms to the profile's depth. That is the cost of touching the chain, and it is paid only by the party who chose to.

## 17 Fees and Conservation

Bitcoin fees consume value, so they appear in the arithmetic rather than being hand-waved. For a one-input partial withdrawal paid from backing,

$$B_n = p + B_{n+1} + f + \delta_A,$$

where  $\delta_A$  is any explicitly retained anchor value. If a separate external input pays the fee, the accounting adjusts and that external source must be explicit. No implementation may silently raise the fee after the burn if the increase changes how much backing leaves the system; an economically relevant change requires a transition whose accounting recomputes exactly.

For one origin lineage, with  $B_v$  the accepted backing remaining,  $S_v$  the live spendable dBTC,  $X_v$  consumed dBTC with an outstanding execution claim, before anything leaves:

$$B_v = S_v + X_v,$$

and after a completed payout and backing-paid fee, both sides fall by the same amount. The invariant is the one that matters to a Bitcoin reader:

### Key Idea

No accepted sequence may leave more redeemable dBTC attributed to an origin than the Bitcoin backing that origin retains under the committed fee and reserve policy.

## 18 Crash Safety

A withdrawal crosses two systems, and the power can fail between any two steps. The stages are: construct the intent; construct the exact transaction candidate; burn; durably retain the completion evidence; derive the fulfillment material; open constrained authority; sign exactly the committed transaction; durably retain it; broadcast; observe settlement; publish the successor if partial.

Two rules make that safe.

**No second candidate after the burn.** Once the burn commits an exact transaction body, recovery either resumes or re-emits the identical committed transaction, or fails closed. It may not use the same burn to construct a different destination, amount, or successor.

**No unsafe remind.** Failing to observe the Bitcoin transaction does not automatically recreate spendable dBTC while a valid signed transaction may still exist. Any recovery or refund transition must prove a mutually exclusive state under which the original release can no longer take effect.

Locally, one vault generation  $(v, n)$  is a unique active key: Class K serialises execution against it or uses an equivalent atomic guard, and a second request against an already committed generation fails closed. That is concurrency control, not a new authority, and it does not replace DSM state validity.

### Bitcoin Comparison

A wallet that crashes mid-signing can usually just try again, because Bitcoin arbitrates conflicting attempts by ordering.

Here the burn has already happened on the DSM side, so “try again differently” is exactly what must be impossible. Recovery is pinned to the transaction the burn committed to.

#### Architectural consequence:

The crash cases are decidable rather than probabilistic: resume the exact committed artifact, or fail closed. There is no window in which a crash lets a second, different withdrawal exist.

#### Tradeoff / boundary:

Fail-closed is a real outcome. A holder can end up needing recovery material to complete a withdrawal that already consumed its dBTC, and the specification requires durable retention at each stage precisely because of that.

## Part IV

# Domains, Compromise, and Assessment

Where online and offline dBTC differ, what a device compromise actually means, how this compares to everything else, and what is not claimed.

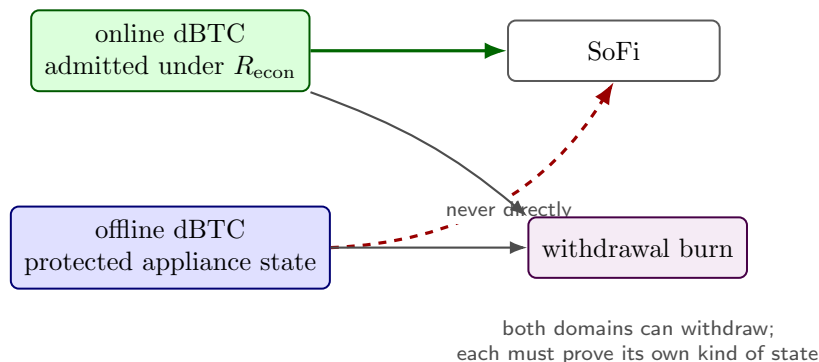
## 19 Online, Offline, and the SoFi Boundary

dBTC lives in whichever DSM custody domain its holder is using, and the two domains are kept apart on purpose.

**Online dBTC** is governed by the canonical economic root  $R_{\text{econ}}$ . A withdrawal from online dBTC proves the burn against that root: the balance and provenance leaf existed in the predecessor root, the claimant had authority, the exact quantity was removed, the resulting root is canonical, the position is admitted under the online root machinery, and the completion evidence binds the intent. A local wallet cache is not a substitute for proof of canonical state.

**Offline dBTC** is a separately allocated bearer domain using the enrolled-appliance and protected-state machinery. A withdrawal from an offline allocation proves that the allocation is live, the appliance authority is genuine, the protected state and counter relation is current, the amount is consumed exactly once, the successor protected state recomputes, and the intent is bound into the completion evidence.

Neither may impersonate the other. An offline burn proof is not an admitted economic root merely because both carry dBTC, and an online balance proof is not a protected offline allocation. Sovereign Finance operates only on admitted online state; an offline allocation does not become SoFi-spendable liquidity because the asset happens to be Bitcoin-backed.



Movement between supported custody domains happens only through the canonical DSM transition defined for that movement. No dBTC-specific bridge silently merges them.

### Bitcoin Comparison

Bitcoin has one custody model: a key signs. Whether you are online is a question about broadcasting, not about validity.

dBTC has two declared domains with different proofs and different boundaries, and which one a given holding lives in is visible rather than implicit.

#### Architectural consequence:

Offline bearer transfer is available for the asset without weakening the online settlement domain, because the two are separated in the protocol rather than by convention.

#### Tradeoff / boundary:

The separation is a real constraint on the user: an offline allocation is not immediately usable as

online liquidity, and converting requires the canonical transition rather than an assertion.

## 20 What a Device Compromise Means

dBTC inherits the security boundary of the DSM custody profile actually in use. If an attacker compromises enough of your DSM authority to produce a transition the applicable verifier accepts as a valid burn, then that dBTC is compromised, and the vault must not pretend otherwise.

Conversely, compromising unrelated storage infrastructure or copying unrelated vault metadata does not create a valid burn.

### Key Idea

dBTC does not become safer than DSM by adding Bitcoin, and Bitcoin must not weaken DSM by becoming an independent bearer-authority system. The relationship is inheritance: dBTC ownership security *is* DSM ownership security, with Bitcoin adding only the external collateral and settlement boundary.

## 21 Security Properties

| Property                  | Statement                                                                                                      |
|---------------------------|----------------------------------------------------------------------------------------------------------------|
| Burn-gated release        | If vault authority is opened through the conforming path, a valid DSM consumption of the bound quantity exists |
| No dBTC from metadata     | Public or replicated vault metadata cannot create a positive balance                                           |
| No stale withdrawal       | A state already consumed by a valid successor cannot satisfy the current-state requirement again               |
| Preimage binding          | Substituting $L_n$ , $C_n$ , or $\sigma$ changes the unlock derivation except with negligible probability      |
| Partial conservation      | A partial withdrawal cannot create a successor retaining more than $B_n - p - f$                               |
| Fresh successor authority | A spent parent generation is not the execution authority for the confirmed successor                           |
| Storage non-authority     | Compromise of storage alone creates no valid burn and no valid withdrawal                                      |
| Depositor non-liveness    | After admission, transfer and withdrawal need no depositor approval                                            |

The proof obligations behind them are stated for the existing verification stack: burn uniqueness (one spendable source cannot be consumed by two accepted burns); burn/unlock implication (any accepted unlock implies some valid burn exists); intent binding (changing amount, destination, generation, or successor commitment fails the execution check); successor conservation; no hidden duplicate reserve; and provenance conservation across transfer, split, merge, and burn.

## 22 Relation to Everything Else That Works

Each of these works, each has a boundary, and the design is best understood by where its boundary sits.

**David Chaum's eCash [8], Cashu, Fedimint.** Blind signatures give a bearer token the mint cannot link to its withdrawal — the original digital cash, and still the cleanest bearer privacy anyone has built. Its boundary is the mint: the mint holds the backing, its database is the double-spend authority,

a compromised mint prints, and redemption needs it online. dBTC keeps the bearer property and the offline capability with no mint: backing is a published autonomous vault, the double-spend authority is DSM, and redemption is a broadcast. What it does not have is Chaum’s blinding.

**Lightning** [9]. Channels between static participants, settled by broadcast inside timeouts. Its boundary is the channel: on-chain setup, locked liquidity, routing, watchtowers, and a counterparty who can hurt you if you miss a window. dBTC has no channels, no timeouts, no routing, and no counterparty liveness requirement — and no sub-satoshi granularity or decade of deployment either.

**Statechains** [10], **Mercury Layer** [11]. Signing authority over a whole UTXO transfers off-chain, but because software deletion is unverifiable the design needs an entity holding a key share, and Mercury runs that share handling in an enclave to make the promise credible. dBTC removes the entity: nobody holds a share of anyone’s key, because release is gated on a state transition rather than on a deletion. And transfers are arbitrary amounts rather than whole UTXOs, with no decrementing timelocks and no coin lifetime.

**Ark** [12]. A pre-signed tree over one shared output, built in rounds with an operator whose co-signature every payment needs, with leaves that expire. dBTC has no shared output among strangers, no operator, no rounds, no co-signature, and no expiry.

**Wrapped and federated Bitcoin.** A custodian or federation holds the coins and issues a claim elsewhere. The boundary is total. dBTC has no custodian and no federation, and the claim on the coin is a state the holder owns rather than an entry in someone’s database.

|                        | Bearer | Offline pay | Sub-dust transfer | Third party in a payment | What a compromise of it costs |
|------------------------|--------|-------------|-------------------|--------------------------|-------------------------------|
| eCash, Cashu, Fedimint | yes    | yes         | yes               | the mint                 | every coin at the mint        |
| Lightning              | no     | no          | yes               | counterparty, watchtower | the channel                   |
| Statechain             | yes    | no          | no                | the entity               | every coin it touched         |
| Ark                    | no     | no          | no                | the operator             | the tree                      |
| Wrapped / federated    | no     | no          | yes               | custodian                | everything                    |
| DSM-native dBTC        | yes    | yes         | yes               | none                     | the holder’s own state        |

## 23 What Is Not Introduced, and What Is Not Claimed

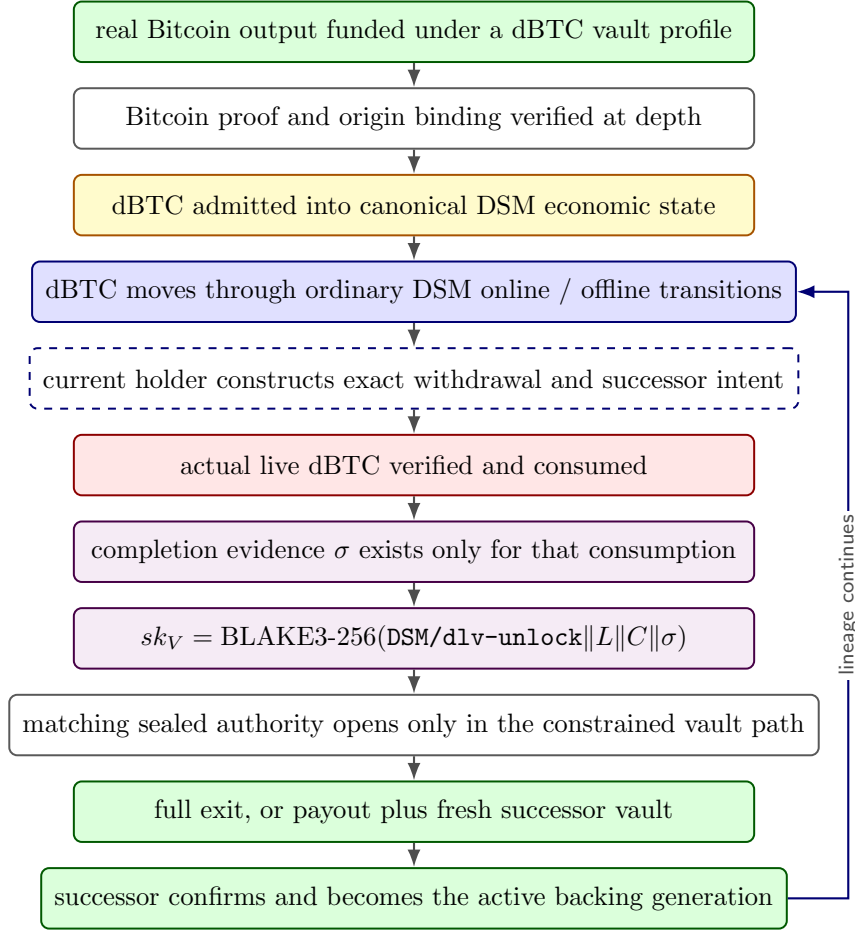
### Deliberately not introduced

No Bitcoin threshold-signing federation. No vault custodian committee. No dBTC validator set. No global dBTC ledger. No mint that approves withdrawals. No storage-node economic voting. No second double-spend database. No requirement that the depositor stay online. No wall-clock settlement validity. No dBTC-only ownership model replacing DSM. No dBTC-only hardware appliance. And no rule that knowing a preimage constitutes ownership.

### Deliberately not claimed

That DSM can prevent a Bitcoin reorganisation deeper than the chosen confirmation assumption. That dBTC survives compromise of the DSM authority needed to produce a valid burn. That storage unavailability cannot delay retrieval of vault material. That malformed Bitcoin transactions can be made safe by application convention. That a reusable parent scalar may be exported after a partial burn without constraining what it signs. That a depositor refund path may coexist with outstanding live dBTC absent a mutually exclusive condition. That testnet or signet shortcuts imply mainnet security. Or that conformance follows merely because the primitives exist in a repository.

## 24 The Complete Causal Architecture



## 25 Final Invariant

The complete safety relation, compactly:

$$\begin{aligned}
 & \text{LiveDbtcBefore}(W) \wedge \text{ValidAuthority}(W) \\
 & \wedge \text{ValidOrigin}(W, V_n) \wedge \text{ValidBurn}(W, \sigma) \\
 \text{BitcoinRelease}(V_n, W) \implies & \wedge sk_{V_n} = \text{BLAKE3-256}(\text{DSM}/\text{dlv-unlock}\|L_n\|C_n\|\sigma) \\
 & \wedge \text{SHA256}(sk_{V_n}) = h_{f,n} \\
 & \wedge \text{BitcoinBody}(W) = T_n \wedge \text{ValueConserved}(T_n).
 \end{aligned}$$

For a full withdrawal  $V_n \rightarrow \text{BTC}$  and the lineage terminates for that backing. For a partial withdrawal  $V_n \rightarrow \text{BTC} + V_{n+1}$ , with the successor carrying the remainder under fresh authority.

The dBTC is never the Bitcoin key.  
 The Bitcoin key is never the ownership proof.  
 The preimage is never sufficient by itself.  
 state  $\rightarrow$  verification  $\rightarrow$  consumption  $\rightarrow$  completion proof  $\rightarrow$  fulfillment  $\rightarrow$  execution

## 26 Conclusion

dBTC does not need a separate distributed authority to prevent its own double spending. The value already exists inside DSM’s state machine. A holder who wants to withdraw must prove possession of that real state and irreversibly consume the required quantity under the same validity machinery that governs every other DSM economic transition.

That consumption produces the completion evidence the vault needs. The vault’s domain-separated fulfillment relation derives the matching preimage only from the correct lock, parameters, and completion commitment. The resulting Bitcoin authority is settlement machinery for one exact authorised withdrawal. A partial withdrawal spends the current vault into a payout and a fresh successor; a full withdrawal terminates the generation. In neither case does the storage layer become a custodian, signer, mint, or validator.

The construction therefore preserves the architecture DSM and Sovereign Finance already share: state is authority; validity is independently verified; consumed state does not remain spendable; storage is non-authoritative; and external execution occurs only after the deterministic predicate holds.

## References

- [1] B. Ramsay. *dBTC: Bitcoin-Backed DSM State — A Normative Specification for Consumption-Gated DLV Fulfillment, Constrained Bitcoin Release, and Successor Vaults* (DSM-NATIVE dBTC V1). Irrefutable Labs Inc., September 2026.
- [2] B. Ramsay. *Deterministic State Machine: A Concise, Post-Quantum Specification*. Irrefutable Labs Inc.
- [3] B. Ramsay. *Software Authority, Hardware Identity*. Irrefutable Labs Inc., 2026.
- [4] B. Ramsay. *Deterministic State Machines as Guarded Linear Constraint Systems*. Irrefutable Labs Inc., 2026.
- [5] B. Ramsay. *SoFi: Sovereign Deterministic Finance*, revision 5. Irrefutable Labs Inc., August 2026.
- [6] B. Ramsay. *Bitcoin Native, Off-Chain, and Offline — dBTC: A High-Level Explainer for Bitcoin Readers* (LINEAGE BOUND V1). Irrefutable Labs Inc., September 2026.
- [7] S. Nakamoto. *Bitcoin: A Peer-to-Peer Electronic Cash System*. 2008.
- [8] D. Chaum. Blind signatures for untraceable payments. *CRYPTO 1982*.
- [9] J. Poon, T. Dryja. *The Bitcoin Lightning Network*. 2016.
- [10] R. Somsen. Statechains: non-custodial off-chain Bitcoin transfer. 2018–2019.
- [11] CommerceBlock. Mercury Layer: statechain implementation with SGX-enclave key-share handling. 2024.
- [12] Bitcoin Optech. Ark protocol, topic page; Ark Protocol, VTXOs and covenant-less Ark. 2026.